NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC4020 Data Analytics & Mining
Project 2

Group 4

Team Member	Matriculation Number
Lee Choonggi	U2320667H
Sih Jia Qi	U2321900L
Ponyuenyong Kritchanat	U2322517E
Yalamanchili Sanjana	U2323273H

Video link: https://youtu.be/LD901vgf7fk

Task 1 Analysis of Symptom Co-occurrence Patterns

1.1 Data Preprocessing

This study uses the Disease-Symptom dataset containing 4920 entries, each with a Disease label and 17 potential symptom columns (*Symptom_1 to Symptom_17*). Preprocessing involves three key steps: symptom cleaning, synonym normalisation, and transaction formulation.

1.1.1 Symptom Cleaning (Text Standardisation)

The initial preprocessing step implemented via (*clean_symptoms* in *task-1.ipynb*) standardises symptom text by removing null entries, converting to lowercase, trimming whitespace, and replacing underscores with spaces for consistent formatting.

1.1.2 Synonym Normalisation

Medical terminology often uses multiple terms for identical symptoms. The (*get_comprehensive_synonym_map* in *task-1.ipynb)* implements an extensive synonym normalisation strategy mapping 131 unique to 80 standardised terms. This consolidation supports the frequency analysis shown in Figure 1, which displays the top 20 standardised symptoms.

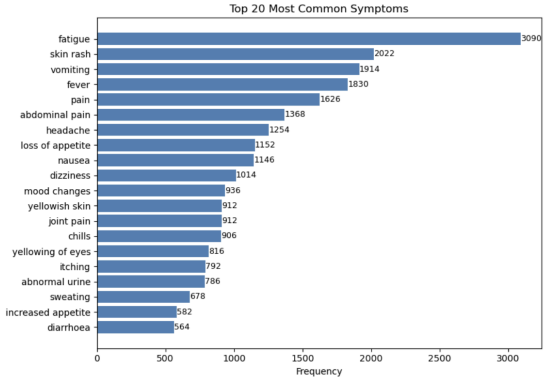


Figure 1: Top 20 most common symptoms

1.1.3 Transaction Formulation and Data Augmentation

The (*prepare_transactions* in *task-1.ipynb*) converts the wide-format dataset into transaction format, with each disease represented by its associated symptoms. To address the limited size (41 diseases) and enhance pattern robustness, (*augment_transactions* in *task-1.ipynb*) applies stochastic augmentation by creating mirrored copies with a 5% symptom dropout. For example, {fever, cough, headache, fatigue, nausea} → {fever, headache, fatigue, nausea}. This process doubles the transaction count to 82 while preserving core symptom relationships.

1.2 Apriori Algorithm Implementation

The Apriori algorithm (Agrawal & Srikant, 1994) was selected for this analysis due to its effectiveness in discovering frequent itemsets and generating association rules.

1.2.1 Core Algorithm Implementation

The algorithm begins with (*generate_frequent_itemsets* in *task-1.ipynb*), which computes support for each individual symptom, using the formula: $Support(X) = \frac{Number\ of\ Transactions\ containing\ X}{Total\ number\ of\ Transactions}$ where X represents an itemset. Candidate generation is then performed by (*generate_candidates* in *task-1.ipyn*b),

which creates k-itemsets by joining pairs of (k-1)-itemsets that share their first k-2 items. A crucial pruning step, implemented in (*prune_candidates* in *task-1.ipynb*), eliminates any candidate whose (k-1)-subsets are not all frequent. The process iterates until no new frequent itemsets remain.

1.2.2 Association Rule Generation

The (*generate_association_rules* in *task1.ipynb*) transforms frequent itemsets containing two or more items into association rules of the form $A \rightarrow B$, where A represents the antecedent (a set of symptoms that implies another symptom) and B represents the consequent (the implied symptoms). For each frequent itemset, the function calculates three key metrics for each rule:

$$Confidence(A \rightarrow B) = \frac{Support(A \cup B)}{Support(A)}, \; Lift(A \rightarrow B) = \frac{Confidence(A \rightarrow B)}{Support(B)},$$
$$Support(A \rightarrow B) = Support(A \cup B)$$

Confidence measures conditional probability, and a lift value greater than 1 indicates positive correlation, with higher values suggesting stronger associations. The support of a rule represents the proportion of transactions where both antecedent and consequent appear together.

1.2.3 Parameter Selection and Optimisation

A minimum support threshold of 0.10 was applied, requiring patterns to occur in at least 9 diseases (10% of 82 augmented transactions). A minimum confidence of 0.50 was set, retaining only rules with at least 50% conditional probability. Lower thresholds risk spurious patterns, while stricter thresholds may miss clinically relevant associations in this relatively small medical dataset.

1.3 Results

1.3.1 Frequent Itemsets Discovery

The algorithm identified 70 frequent itemsets that met the 10% support threshold, comprising 19 individual symptoms (itemset size = 1), 34 symptom pairs (itemset size = 2), 16 symptom triplets (itemset size = 3) and 1 four-symptom combination (itemset size = 4).

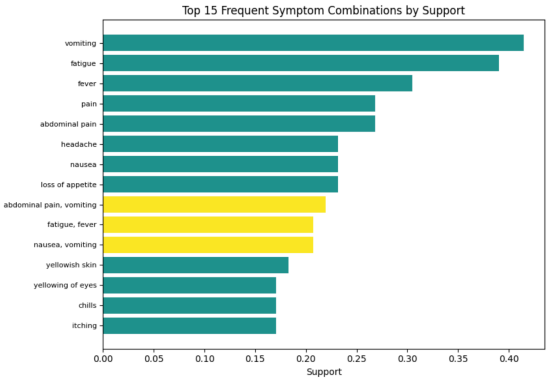


Figure 2: Top 15 frequent symptom combinations by support

Figure 2 shows the top 15 most frequent symptoms. The most prevalent individual symptoms include vomiting (support = 0.4146), fatigue (support = 0.3902), and fever (support = 0.3049), reflecting their widespread occurrence across multiple disease conditions. The discovery of multi-symptom itemsets (up to size 4) demonstrates that certain symptom combinations consistently co-occur in disease presentations.

1.3.2 Association Rules Analysis

Table 1: Top 5 Association rules

Association Rule	Support	Confidence	Lift
{headache, nausea} → {vomiting}	0.1098	1.0000	2.4118
{chills, fatigue} → {fever}	0.1220	1.0000	3.2800
{loss of appetite, yellowish skin} → {yellowing of eyes}	0.1341	1.0000	5.8571
{yellowing of eyes, yellowish skin} → {loss of appetite}	0.1341	1.0000	4.3158
{yellowish skin, vomiting} → {abdominal pain}	0.1341	1.0000	3.7273

112 association rules were generated with average confidence 72.76%, average support 12.47%, and average lift 3.33. The high average lift indicates discovered rules represent genuine symptom associations rather than random co-occurrence. Table 1 presents the top five association rules derived from symptom co-occurrence patterns. Each rule has perfect confidence and high lift values of 2.41-5.86, showing strong non-random relationships. Rules 3 to 5 highlight a hepatic cluster, where symptoms such as yellowish skin, yellowing of eyes, and loss of appetite frequently co-occur and signal liver dysfunction. Rule 5 also links vomiting and abdominal pain, which are common in gastrointestinal or hepatic conditions. In contrast, Rules 1 and 2 capture infection-related clusters, combining symptoms like headache, nausea, chills, and fever, often in viral or systemic illnesses. Support values around 11-13% indicate these patterns recur across multiple diseases.

Task 2 Mining Cancer Feature Patterns

2.1 Data Preprocessing

This study uses the Wisconsin Breast Cancer dataset with preprocessing steps include removal of irrelevant columns, mapping diagnosis labels ("M"=1, "B"=0), and feature normalisation using the Standard Scaler: $z = \frac{(x - \mu)}{\sigma}$ where x is the feature value, μ is the mean, σ is the standard deviation across all samples. Normalisation prevents features with large ranges (e.g., *area_mean*: 143.5-2357.0) from dominating smaller ones (e.g., *smoothness_mean*: 0.05-0.16). To convert numerical features into categorical sequences for pattern mining, three sequence generation strategies were explored: Z-Score Ranking, Mutual Information Ranking, and Discretisation-Based Binning.

2.1.1 Z-Score Ranking

The Z-score ranking method (*make_sequence_zscore* in *task-2.ipynb*) identifies most abnormal features for each individual patient. It computes absolute z-score deviations to quantify how far each feature diverges from the population mean. For each patient, features are ranked by their deviation magnitude, and the top 10 features are selected to form the sequence. Features with similar z-scores (within a tie threshold of 0.01) are grouped into itemsets to represent concurrent abnormalities. Each selected feature is labeled as "high" or "low" depending on whether its z-score is positive or negative respectively.

2.1.2 Mutual Information (MI)

Table 2: Top10 Features Ranked by Mutual Information

Rank	Feature	MI Score
1	perimeter_worst	0.472
2	area_worst	0.464
3	radius_worst	0.451
4	concave points_mean	0.439
5	concave points_worst	0.436
6	perimeter_mean	0.402
7	concavity_mean	0.375
8	radius_mean	0.362
9	area_mean	0.360
10	area_se	0.341

The MI ranking method (*make_sequence_mi* in *task-2.ipynb*) takes a global approach by prioritising features based on their predictive relevance to the diagnosis label. Unlike Z-score ranking, which generates patient-specific orderings, MI uses a fixed feature order applicable to all patients. MI scores are computed using scikit-learn's *mutual_info_classif* function (with random_state = 42 for reproducibility). As shown in Table 2, the top ranked features include *perimeter_worst* (MI = 0.472) and *area_worst* (MI = 0.464). For sequence generation, the same top 10 features appear in the same order for every patient. Only the directional labels ("high" or "low") vary between patients based on whether their individual feature values are above or below the median. This method sacrifices personalized ordering to evaluate whether malignant and benign cases exhibit consistent patterns aligned with globally important features.

2.1.3 Discretisation-based Binning

The discretisation-based binning method (*make_sequence_binned* in *task-2.ipynb*) converts continuous features into three ordinal bins using one of three strategies: quantile (equal number of patients per bin), uniform (equal value ranges), or k-means (cluster-based). Similar to the Z-Score ranking, features are first ranked by absolute z-score per patient, and itemsets are formed for features with similar scores. Each feature is then assigned with a ternary label: "low" (bin 0), "med" (bin 1), or "high" (bin 2).

Among these strategies, quantile binning was chosen as the primary method due to its balanced representation across patients and superior performance in pattern discovery (Section 2.2.2). Although combining patient-specific ranking with discretisation increases computational workload compared to global methods like MI, the extra cost is justified as quantile binning preserves patient-specific differences and produces more diverse sequential patterns. Therefore, this makes quantile binning the most effective method for capturing diagnostic patterns.

2.2 Sequential Pattern Mining

With patient sequences generated, we move on to discover frequent sequential patterns that distinguish malignant from benign cases. Two sequential mining algorithms are compared: Generalised Sequential Pattern (GSP) and Prefix-Projected Sequential Pattern Mining (PrefixSpan). Before the comparison, sensitivity tests were conducted to determine the optimal parameters for our algorithms.

2.2.1 Sequence Generation Strategies Comparison

Table 3: Performance Comparison of Sequence Generation Methods

Method	Malignant Patterns	Benign Patterns	Avg. Support (Mal)	Avg. Support (Ben)
Binned (Quantile)	49	38	0.259	0.243
Z-score	49	40	0.259	0.246
Mutual Information	27	25	0.777	0.868

Table 3 highlights a trade-off between patient-specific personalisation and global pattern discovery. The Binned (quantile) and Z-score methods, which use individualised feature rankings, produced 87 and 89 patterns respectively, with lower average support (~24–26% of patients), capturing rarer but meaningful variations. In contrast, the Mutual Information (MI) method yielded fewer patterns (52) with higher support (0.78 malignant, 0.87 benign), emphasising common diagnostic trends but reducing sensitivity to individual differences. Based on this balance, the Binned (quantile) method was selected for subsequent analysis.

2.2.2 Binning Strategy Sensitivity

Table 4: Comparison of Binning Strategies Based on Pattern Counts and Average Length

Strategy	Malignant Patterns	Benign Patterns	Avg. Length (Mal)	Avg. Length (Ben)
Uniform	38	37	1.211	1.243
Quantile	49	38	1.490	1.184
K-means	32	36	1.281	1.250

Table 4 shows that quantile binning outperforms uniform and k-means in both pattern discovery and complexity. It identifies 49 malignant patterns versus 38 (uniform) and 32 (k-means). By dividing patients into equal thirds (~189 per bin), quantile maintains balanced support across intensity levels, enabling GSP to capture rare but clinically significant progression patterns.

Quantile also yields longer malignant sequences (avg. 1.49 itemsets) compared to 1.21 (uniform) and 1.28 (k-means), while benign sequences remain shorter (1.18–1.25). The larger malignant-benign gap (0.31 itemsets) highlights stronger diagnostic discrimination. Based on these results, quantile binning was selected for subsequent pattern mining.

2.2.3 Support Threshold Sensitivity

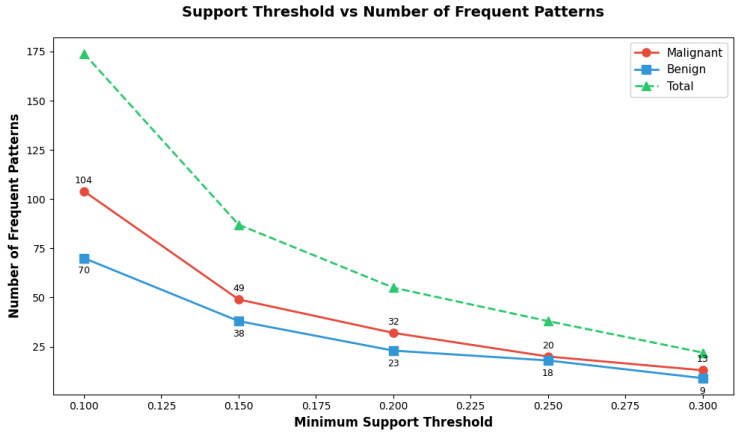


Figure 3: Number of frequent patterns across varying minimum support thresholds

Figure 3 shows that a minimum support threshold of 0.15 balances coverage and efficiency. Lower thresholds (e.g., 0.10) yield too many patterns (104 malignant, 70 benign), adding noise and computation, while higher thresholds (0.25–0.30) reduce counts to 32 and 23 total patterns respectively, risking loss of meaningful malignant patterns. At 0.15, the system captures 49 malignant and 38 benign patterns, keeping results both relevant and manageable.

2.2.4 Feature Importance

Table 5: Most Important Features Weighted by Pattern Support

Malignant Cases			Benign Cases		
Rank	Feature	Weight	Rank	Feature	Weight
1	perimeter_mean_high	2.241	1	radius_mean_low	0.860
2	radius_mean_high	2.047	2	perimeter_mean_low	0.843
3	radius_worst_high	2.047	3	perimeter_worst_low	0.644
4	perimeter_worst_high	1.892	4	radius_worst_low	0.622
5	area_mean_high	1.858	5	area_mean_low	0.577
6	area_worst_high	1.288	6	texture_worst_low	0.496
7	concave points_worst_high	0.462	7	texture_mean_low	0.485
8	concave points_mean_high	0.439	8	concave points_worst_low	0.381
9	radius_se_high	0.425	9	concavity_worst_low	0.350
10	area_se_high	0.387	10	compactness_mean_low	0.339

Table 5 shows an analysis of the most important features extracted from the sequential patterns discovered using the binned (quantile) approach, along with the GSP mining algorithm. From here, we identified

which features most strongly contribute to differentiating malignant from benign cases by aggregating feature importance weighted by pattern support. For each class, we added up the support of every pattern that contained a given item, so items that show up often inside frequent patterns score higher.

For malignant cases, the top features indicate high values in tumor size and shape. The highest-ranked feature is *perimeter_mean_high*, with the largest cumulative support score of 2.241. Other strong features include *perimeter_worst_high* and *area_mean_high*, which suggests that larger perimeter and area are often linked with malignant tumors. Features related to concavity and radius such as *concave points_worst_high* and *radius_se_high* also contribute greatly.

On the other hand, benign case features show an opposite trend where lower measurements are the most important. The highest weights include *radius_mean_low* and *perimeter_mean_low* with support scores of 0.860 and 0.843 respectively. Other features include *perimeter_worst_low*, *radius_worst_low*, and *area_mean_low* which reflects how smaller tumor sizes are typically associated with benign diagnoses.

2.2.5 Algorithm Validation: GSP vs. PrefixSpan

To ensure the reliability of discovered patterns, we performed a comparison between Generalised Sequential Pattern (GSP) and PrefixSpan algorithms using the binned (quantile) sequence approach.

Table 6: Algorithm Performance Comparison on Binned (Quantile) Sequences

Algorithm	Malignant Patterns	Benign Patterns	Total
GSP	49	38	87
PrefixSpan	4	0	4

Table 6 reveals that GSP produced 49 distinct malignant patterns and 38 benign patterns which totals to 87 patterns overall. However, PrefixSpan identified only 4 malignant patterns and did not find any frequent patterns in the benign group, totaling up to only 4 patterns. This significant difference in the number of patterns is due to the way each algorithm interprets sequential data. GSP is able to find both simple (single-feature) and composite (multi-feature) patterns, while PrefixSpan focuses more on multi-step relationships that are less frequent, especially in benign cases.

Table 7: Top 5 Patterns Discovered by GSP and PrefixSpan

GSP - Malignant			GSP - Benign		
Rank	Pattern	Support	Rank	Pattern	Support
1	{perimeter_mean_high}	0.491	1	{concave points_worst_low}	0.381
2	{perimeter_worst_high}	0.472	2	{concavity_worst_low}	0.350
3	{radius_mean_high}	0.472	3	{compactness_mean_low}	0.339
4	{radius_worst_high}	0.472	4	{texture_worst_low}	0.336
5	{concave points_worst_high}	0.462	5	{perimeter_mean_low}	0.328

PrefixSpan - Malignant			PrefixSpan - Benign
Rank	Pattern	Support	*No patterns found*
1	radius_worst_high → perimeter_worst_high	0.198	
2	radius_mean_high → perimeter_mean_high	0.160	
3	area_worst_high → perimeter_worst_high	0.151	
4	area_mean_high → perimeter_mean_high	0.146	

Table 7 shows the top 5 patterns discovered by each algorithm. The highest supported patterns mined by GSP in malignant cases contain multiple measurements in tumor perimeter and radius with *perimeter_mean_high* achieving the highest support of 0.491. The most significant benign patterns from GSP include lower values in concave points, concavity, and compactness. This reflects the smoother and less aggressive tumor characteristics, with *concave points_worst_low* having the highest support of 0.381. On the other hand, PrefixSpan mainly mined multi-step patterns in malignant cases. The top pattern, *radius_worst_high* → *perimeter_worst_high* (support=0.198) indicates a sequential relationship where high radius measurements are followed by high perimeter values. Notably, no frequent patterns were

found in benign sequences. This suggests that malignant cases display more consistent sequential relationships among tumor features.

Task 3 Open Advanced Tasks

Hospital triage often struggles with long wait times, heavy manpower demands, and inconsistent decision-making, especially when staff must rapidly process patients using static questionnaires and limited information. These bottlenecks slow down care and increase the risk of misrouting patients to the wrong departments. Our system addresses this by turning triage into a dynamic, evidence-driven process that updates predictions as symptoms are confirmed and provides an automated, consistent referral pathway, reducing workload, improving accuracy, and supporting faster patient flow. The demo version of the solution is linked as follows: https://sc4020-project-2-task-3.streamlit.app/ .

3.1 System Overview

The UI collects symptoms and sends them to the backend, which filters options, retrains the model, and updates predictions in real time using the DataManager and ModelManager. Once confident, the system triggers an LLM verifier to choose the appropriate hospital department. The final diagnosis, precautions, and referral are then returned to the user.

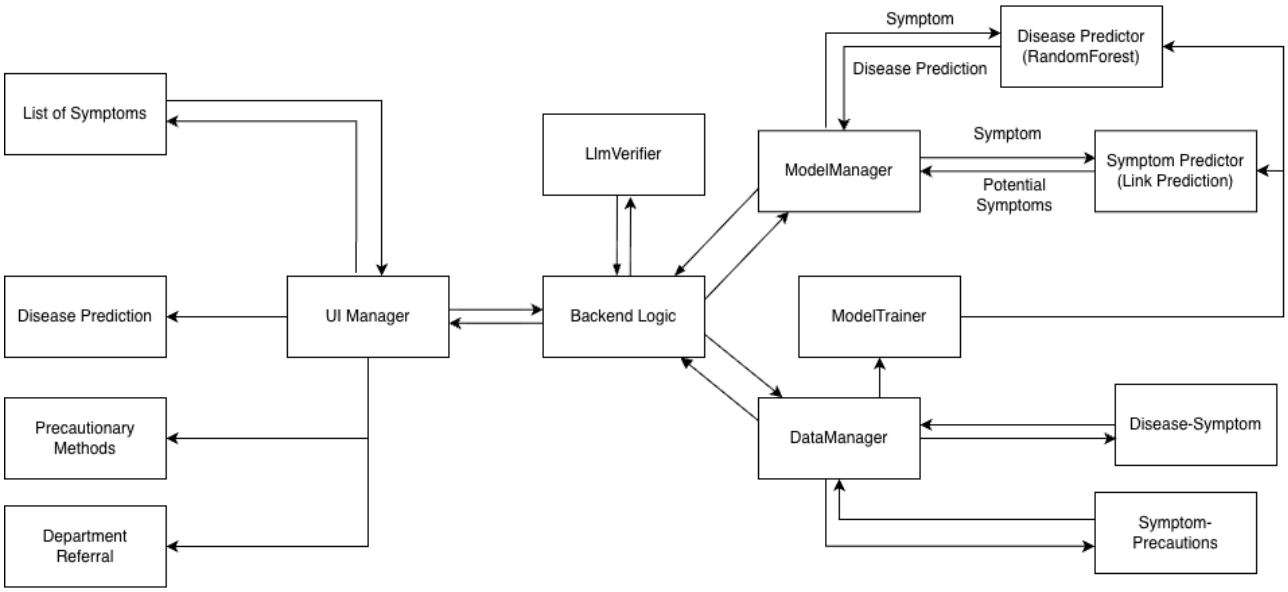


Figure 4: Architecture of the Progressive Diagnostic Triage Model

3.2 Model Training and Logic Flow

The system employs a confirm-time training mechanism in which a Random Forest classifier is retrained at each stage of symptom confirmation. This ensures that predictions remain aligned with the patient's evolving evidence path rather than relying on a static model. At every step, the model evaluates a synthetic query vector where all confirmed symptoms are marked present, producing a probability distribution across candidate diseases. Predictions are displayed only when they exceed a 0.70 probability threshold, which keeps results interpretable and avoids overfitting to weak signals. If no disease meets this threshold, the system applies a progressive relaxation strategy, gradually broadening inclusion criteria (e.g., $\geq$ k-1 or $\geq$ k-2 neighbors) until a meaningful prediction emerges. This guarantees that inference remains possible while preserving the logical coherence of symptom filtering.

Beyond prediction, the system integrates a next-symptom suggestion component designed as a conditional co-occurrence predictor. Formulated as a link prediction task, it estimates the likelihood that an unselected symptom would appear given the current evidence set, based on distributions within the filtered pool. This approach avoids arbitrary or global "most common symptom" prompts, instead offering context-aware recommendations that reflect the patient's unique presentation.

Finally, once a confident disease prediction is reached, an LLM-driven referral verification step determines the most appropriate department. Unlike static rule-based mappings, the LLM considers the full evidence path and symptom context, producing a more reliable and flexible referral mechanism that adapts to the nuances of each case. Together, these components create a system that is not only technically robust but also clinically interpretable, functioning as a dynamic reasoning loop rather than a fixed diagnostic checklist.

Table 8: Results of Random Forest Model

Total	Confident cases (≥ 0.70)	Overall accuracy	Confident Cases accuracy	Avg.% symptoms used to reach hit ($\pm$ SD)
984	932	0.9472	1.0000	$0.3707 \pm$ *0.1107*

As shown in Table 8, the system achieved high reliability, correctly identifying 94.7% of test cases, with all confident predictions being accurate. On average, it required only 37% of a case's symptoms (±11%) to reach a correct diagnosis.

3.3 Innovations and Rationale

The project introduces four interlocking innovations that make the approach both adaptive and interpretable.

First, **sequential questioning of critical symptoms** mimics real-life triage, ensuring that urgent indicators such as chest pain influence predictions earlier than mild symptoms like cough, while also reflecting the clinical need to prioritise urgent cases.

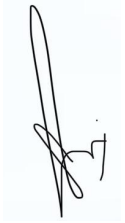
Second, the **pool-limited selection mechanism** restricts users to symptoms that remain statistically plausible within the filtered dataset, eliminating contradictory combinations, preserving logical consistency, and improving scalability as the pool narrows with each confirmation.

Third, the **evidence-conditioned suggestion engine** replaces generic recommendations with symptom proposals derived from conditional prevalence, dynamically tailoring prompts to the user's current presentation by analysing co-occurrence patterns within the filtered pool. This transforms the assistant from a static checklist into a responsive questioner that adapts to evolving evidence.

Finally, **runtime random-forest retraining** converts each round of symptom confirmation into an interpretable update step, recalibrating probabilities in real time based on verified symptoms rather than relying on a fixed model.

Together, these innovations enable the system to function as a reasoning loop rather than a one-shot predictor, progressively refining both hypotheses and recommendations as new evidence emerges.

SC4020 Project 2 Group 4 Team Members Contributions Form

No.	Name	Contribution (%)	Justification
1	Yalamanchili Sanjana 	25%	Task 1 Analysis of Symptom Co-occurrence Patterns - Code creation - Slides creation - Video presentation - Report Writing
2	Sih Jia Qi 	25%	Task 2 Mining Cancer Feature Patterns - Code creation - Slides creation - Video presentation - Report Writing
3	Ponyuenyong Kritchanat 	25%	Task 2 Mining Cancer Feature Patterns - Code creation - Slides creation - Video presentation - Report Writing
4	Lee Choonggi 	25%	Task 3: Open Advanced Tasks - Code creation - Slides creation - Video presentation - Report Writing